

# SIMATS ENGINEERING

## ASSESSMENT TOOL 2

**COURSE NAME: COMPUTER NETWORKS**

**COURSE CODE: CSA07**

**Name: Y. Geetha Reddy**

**Reg no: 192572053**

---

### COURSE OUTCOME COVERED

**CO5:** Measure network performance using key metrics with simulation tools.  
(BL4, BL5)

*Assessment Tool Weightage: 30%*

---

### Annexure A

## Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.

### Network Throughput and Packet Loss

Given:

- Total data received = 8000 bits
- Time = 4 seconds
- Packets sent = 200
- Packets received = 180

### Pseudocode:

ALGORITHM CalculateNetworkPerformance

BEGIN

INPUT totalDataReceived

INPUT time

INPUT totalPacketsSent

INPUT totalPacketsReceived

// Validate inputs

IF time <= 0 THEN

    DISPLAY "Error: Time must be greater than zero."

    STOP

END IF

```
IF totalPacketsSent <= 0 THEN
    DISPLAY "Error: Total packets sent must be greater than zero."
    STOP
END IF
```

```
// Calculate throughput
throughput = totalDataReceived / time
```

```
// Calculate packet loss
lostPackets = totalPacketsSent - totalPacketsReceived
packetLossPercentage = (lostPackets / totalPacketsSent) * 100
```

```
// Display results
DISPLAY "Network Throughput = ", throughput, " bps"
DISPLAY "Packet Loss = ", packetLossPercentage, "%"
```

```
// Classify network
IF throughput > 1500 AND packetLossPercentage < 10 THEN
    networkStatus = "Efficient"
ELSE
    networkStatus = "Inefficient"
END IF
```

```
DISPLAY "Network Classification = ", networkStatus
```

```
// Complete summary
DISPLAY "----- Network Performance Summary -----"
DISPLAY "Data Received: ", totalDataReceived, " bits"
DISPLAY "Time: ", time, " seconds"
DISPLAY "Packets Sent: ", totalPacketsSent
DISPLAY "Packets Received: ", totalPacketsReceived
DISPLAY "Lost Packets: ", lostPackets
DISPLAY "Throughput: ", throughput, " bps"
DISPLAY "Packet Loss: ", packetLossPercentage, "%"
DISPLAY "Overall Performance: ", networkStatus
```

END

## Calculation:

### Throughput:

$$8000 / 4 = 2000 \text{ bps}$$

### Lost packets:

$$200 - 180 = 20$$

### Packet loss:

$$(20 / 200) \times 100 = 10\%$$

Since throughput is **2000 bps > 1500 bps**, but packet loss is **10% and not less than 10%**, the network is:

### Classification: Inefficient

The question specifically requires both conditions to be satisfied for an Efficient classification.

2. In organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

### **Bandwidth Utilization Monitoring:**

The algorithm should continuously collect bandwidth usage, store it in an array/list, detect utilization above 80%, and recommend switching to a backup link.

### **Pseudocode:**

ALGORITHM MonitorBandwidth

BEGIN

    INPUT numberOfLinks

    CREATE bandwidthUsage[numberOfLinks]

    CREATE linkStatus[numberOfLinks]

    WHILE TRUE DO

        DISPLAY "----- Bandwidth Monitoring -----"

        FOR i = 1 TO numberOfLinks DO

            INPUT bandwidthUsage[i]

            // Validate bandwidth utilization

            IF bandwidthUsage[i] < 0 OR bandwidthUsage[i] > 100 THEN

                DISPLAY "Invalid utilization for Link ", i

                linkStatus[i] = "Invalid"

            ELSE

                IF bandwidthUsage[i] > 80 THEN

                    linkStatus[i] = "Overloaded"

                    DISPLAY "Link ", i, " is overloaded."

                    DISPLAY "Utilization = ", bandwidthUsage[i], "%"

                    DISPLAY "Recommendation: Switch traffic to backup link."

                ELSE

                    linkStatus[i] = "Normal"

                    DISPLAY "Link ", i, " is operating normally."

                    DISPLAY "Utilization = ", bandwidthUsage[i], "%"

            END IF

```

        END IF

    END FOR

    DISPLAY "----- Current Link Status -----"

    FOR i = 1 TO numberOfLinks DO

        DISPLAY "Link ", i, ": ", bandwidthUsage[i], "% - ", linkStatus[i]

    END FOR

    WAIT monitoringInterval

END WHILE

END

```

### **Explanation:**

This algorithm continuously monitors every network link using a WHILE loop and stores bandwidth utilization values in an array. A FOR loop checks each link individually. If utilization exceeds 80%, the link is classified as overloaded and the algorithm recommends moving traffic to a backup link. This helps administrators identify congestion early, distribute traffic efficiently, and prevent performance degradation. Periodic monitoring also allows the system to respond to changing traffic conditions.

3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

This question requires bandwidth, delay, and utilization to be monitored periodically, a link-quality score to be calculated, and the highest-scoring path to be selected. It must also handle overloaded/failed links and automatically select an alternate path.

### **Pseudocode:**

```

ALGORITHM SelectBestNetworkPath

BEGIN

    INPUT numberOfPaths

    CREATE bandwidth[numberOfPaths]

    CREATE delay[numberOfPaths]

    CREATE utilization[numberOfPaths]

    CREATE linkStatus[numberOfPaths]

    CREATE qualityScore[numberOfPaths]

    WHILE TRUE DO

        DISPLAY "----- Network Path Monitoring -----"
    
```

```

FOR i = 1 TO numberOfPaths DO

    INPUT bandwidth[i]

    INPUT delay[i]

    INPUT utilization[i]

    INPUT linkStatus[i]

    // Check whether the link has failed

    IF linkStatus[i] = "FAILED" THEN

        qualityScore[i] = -1

        DISPLAY "Path ", i, " has failed."

        DISPLAY "Path ", i, " will not be selected."

    ELSE

        // Check whether the link is overloaded

        IF utilization[i] > 80 THEN

            qualityScore[i] = -1

            DISPLAY "Path ", i, " is overloaded."

            DISPLAY "Path ", i, " will not be selected."

        ELSE

            // Calculate link quality score

            qualityScore[i] =

                (0.5 * bandwidth[i]) +

                (0.3 * (100 - delay[i])) +

                (0.2 * (100 - utilization[i]))

            DISPLAY "Path ", i,

                " Quality Score = ", qualityScore[i]

        END IF

    END IF

END FOR

// Find the highest quality path

bestPath = -1

highestScore = -1

FOR i = 1 TO numberOfPaths DO

    IF qualityScore[i] > highestScore THEN

        highestScore = qualityScore[i]

        bestPath = i

    END IF

END FOR

```

```
// Select path

IF bestPath = -1 THEN

    DISPLAY "ALERT: No suitable network path is available."

    DISPLAY "Administrator intervention required."

ELSE

    DISPLAY "Best Communication Path = Path ", bestPath

    DISPLAY "Highest Quality Score = ", highestScore

END IF

// Check for path changes

FOR i = 1 TO numberOfPaths DO

    IF linkStatus[i] = "FAILED" OR utilization[i] > 80 THEN

        DISPLAY "ALERT: Path ", i,

            " is unavailable or overloaded."

    END IF

END FOR

WAIT monitoringInterval

END WHILE

END
```

**Explanation:**

The algorithm periodically collects bandwidth, delay, and utilization for every path and stores them in arrays. A weighted quality score is calculated using these metrics, and the path with the highest score is selected. Failed or overloaded paths are assigned a score of –1, preventing them from being selected. If the current path fails or becomes overloaded, another available path with the highest score is automatically chosen. Alerts inform the administrator about failures or congestion. This improves routing efficiency, reliability, fault tolerance, and adaptability to changing network conditions.

**RUBRICS FOR CALCULATION**

| Criteria | Weight<br>ag e | Level 4 –<br>Exempla<br>ry | Level 3<br>–<br>Proficie<br>nt | Level 2 –<br>Developi<br>ng | Level 1<br>–<br>Beginni<br>ng |
|----------|----------------|----------------------------|--------------------------------|-----------------------------|-------------------------------|
|          |                |                            |                                |                             |                               |

|                           |     |                                                                       |                                           |                              |                                 |
|---------------------------|-----|-----------------------------------------------------------------------|-------------------------------------------|------------------------------|---------------------------------|
| Problem Understanding     | 20% | Identifies all inputs, outputs, constraints accurately                | Minor missing elements                    | Partial understanding        | Incorrect interpretation        |
| Algorithm Design          | 30% | Complete, correct, and logically sound solution                       | Minor logical gaps                        | Partially correct logic      | Incorrect approach              |
| Use of Control Structures | 20% | Efficient and appropriate use of loops, conditions, functions/modules | Minor inefficiencies                      | Limited or basic usage       | Incorrect or missing constructs |
| Pseudocode Structure      | 10% | Well-structured, clear, consistent format, easy to follow             | Mostly clear with minor issues            | Difficult to follow in parts | Poorly structured and unclear   |
| Completeness              | 20% | Handles all cases; optimized approach                                 | Handles most cases; reasonable efficiency | Handles basic cases only     | Incomplete or inefficient       |